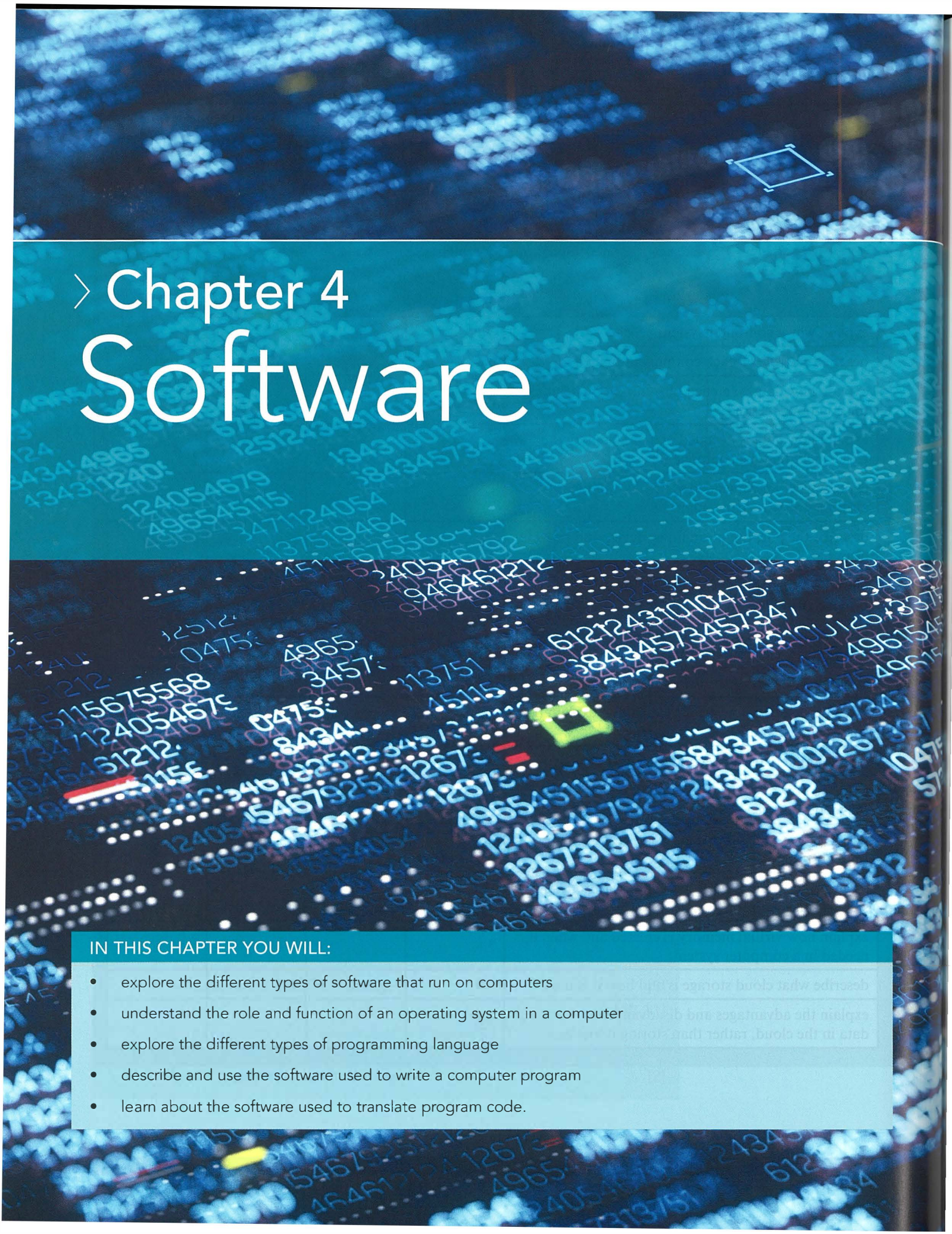


# Chapter 4
# Software

IN THIS CHAPTER YOU WILL:

- explore the different types of software that run on computers
- understand the role and function of an operating system in a computer
- explore the different types of programming language
- describe and use the software used to write a computer program
- learn about the software used to translate program code.

## GETTING STARTED

Work in pairs to make a list of all the different pieces of software that you have used. Can you think of a way to categorise these pieces of software? For example, by what they do or how they are used?

As a class create a mind-map of the different programming languages that you have used, or come across. Discuss whether any of these have specific uses, or do they allow for any type of program to be written?

## DEVELOPMENT OF PROGRAMMING LANGUAGES

The very first programmable digital computer was developed during World War 2 in the United States of America; the Electronic Numerical Integrator and Computer (ENIAC). Its purpose was to perform calculations faster than a human. It allowed the storage of a 10-digit number and could perform addition and subtraction. This computer could only perform the task it was designed to perform.

From this, assembly language was created that allowed a user to write a program that could then be converted into binary code (see binary interpretations in Chapter 1). These languages required the user to state which memory locations were being accessed (for example, store the data in memory location 3, access the data from memory location 4. This takes a lot of expertise to write a long program). Each program had to be written to the assembly language of the computer it was to run on. If you wanted the same program on two computers from different manufacturers, or different versions, then you would have to write the program again in that computer's language.

Fortran was the first significant high-level programming language first released in 1957 by IBM. This was different from previous languages because it was the first language that did not require the user to write in assembly language. It used more user-friendly commands that were then converted into code that could be run. Another benefit of this new language was that it was not machine-dependent, you could write a program on one computer, and run it on an entirely different one.

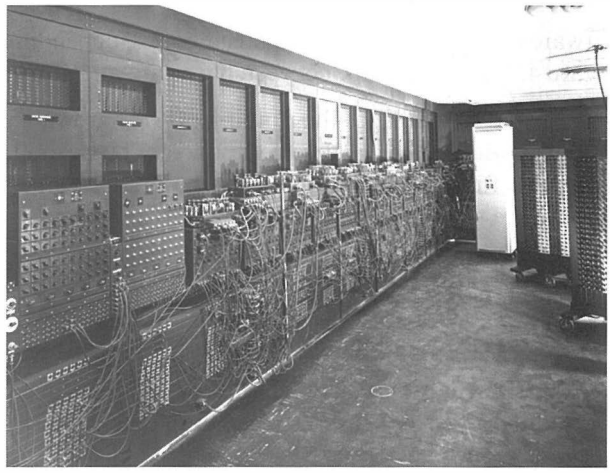


**Figure 4.1:** The ENIAC

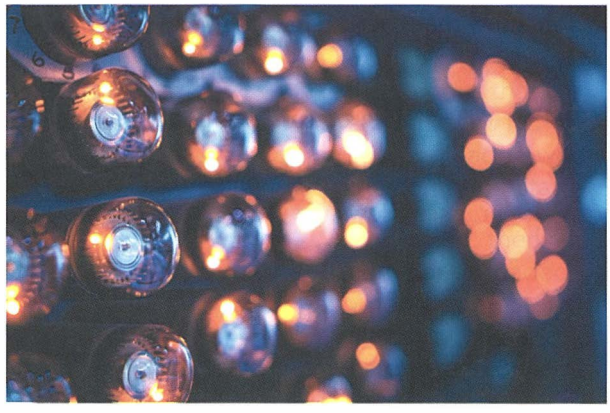


**Figure 4.2:** Close-up of an original computer similar to a Turing machine

### Discussion questions

1 Can you think of any more problems that can arise from only writing programs in assembly language?

2 How has the development of programming languages impacted the use of computers?

# 4.1 Types of software

Computers need **software** to allow the user to do something useful. You will use software each time you use a computer, whether this is a PC, a laptop, tablet or mobile phone. Software is a series of instructions that tell the computer what to do, created using programming languages..

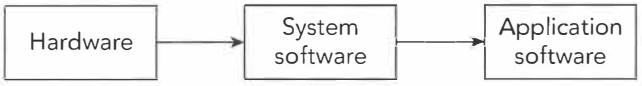



**Figure 4.3:** System software manages the running of hardware; application software runs on top of system software

Software can be put into different categories, depending on their purpose and use (Figure 4.3 shows the link between hardware and software).

**System software** manages the running of the hardware and other software, and allows communication between these components. These are tasks that help the computer perform its functions and improve or maintain its efficiency. Two types of system software are the **operating system (OS)** and **utility programs**.

Examples of utility programs are not required but useful to know. These are programs that maintain the computer system, for example:

- System clean-up-searches for and removes un-used programs and data.
- Defragmentation-rearranges files so that they are located together (contiguously) and that the free spaces is located together.

**Application software** allows the user to perform a task – it provides the services for the user, for example, write a letter, view a website, etc.

Application software have generic names, as well as the brand names. It is important that you know the generic names, some of the common ones are:

- A **word processor** allows the user to create written documents.
- A **spreadsheet** allows the user to perform calculations on data.
- A **database** allows the user to store and search a set of data.
- A **web browser** allows the user to view and navigate websites.

## KEY WORDS

**software:** a series of instructions written in a programming language that performs a function.

**system software:** a type of software that manages the hardware and software in a computer.

**operating system (OS):** a program that manages the computer, allows the user to communicate with the computer, and allows software to be run.

**utility programs:** system software that performs house-keeping activities.

**application software:** software that allows the user to perform a useful task.

**word processor:** a type of application software that allows the user to create text-based documents.

**spreadsheet:** a type of application software that performs calculations on data.

**database:** an example of application software to store and manipulate data.

**web browser:** a piece of software that retrieves and displays web pages.

## COMPUTER SCIENCE IN CONTEXT

PCs, laptops, tablets and smartphones all have application software. The brand names may be different, but the fundamental purposes are the same. For example, you can open a word processor on a PC, a laptop, a tablet and even a smartphone.

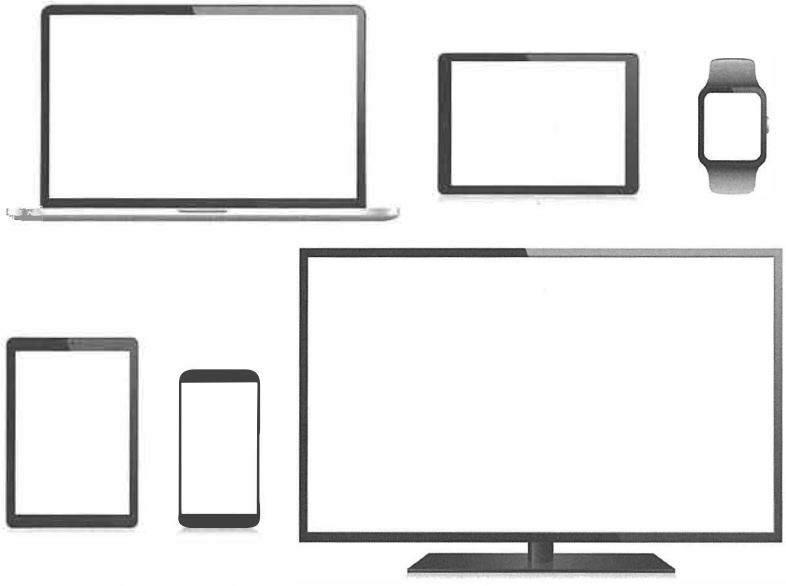


**Figure 4.4:** Digital devices come in many different forms

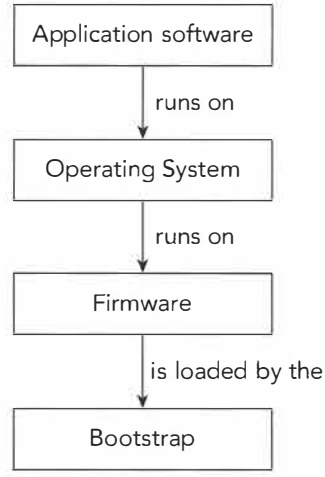



**Figure 4.5:** The software on a computer.

The application software cannot run on a computer without an operating system. The operating system cannot run unless there instructions given to it, these are provided by the **firmware**. The firmware cannot run unless it is told to run, this is done using the **bootstrap** program, which is loaded when the computer first turns on. This process can be represented as shown in Figure 4.5 on the right.

## ACTIVITY 4.1

Log onto a computer, tablet or mobile phone. Make a list of the different types of application software that are installed. Remember not to use brand names.

## KEY WORDS

**firmware:** instructions that are stored in the ROM and are loaded when the computer starts.

**bootstrap:** the first commands loaded when a computer starts, it checks hardware and loads the firmware.

# 4.2 Operating system

An operating system (OS) is an example of system software, you may be familiar with brands such as Windows, Linux and macOS for PCs and laptops. Other computers also have operating systems, for example, a mobile phone may have an Android OS, or iOS. The overall purpose of these operating systems is to allow you, the user, to interact with the hardware.

The OS has many different roles that are described in the following sections.

# Providing an interface

The OS allows the user to enter data, and outputs data to the user, all through the **interface**. There are different types of interface:

- **Graphical User Interface (GUI)** has windows, icons, menus and pointers (WIMP). It is what you will be familiar with from PCs, mobile phones, etc. This is most useful for novices because it is often intuitive. An example is Windows.
- A **command line interface** requires the user to enter text as commands. This is most useful for expert users because the commands entered must be exact. An example is Linux.
- A **natural language interface** allows the user to enter any text commands, or speak commands. The OS then analyses what has been entered and performs the action.

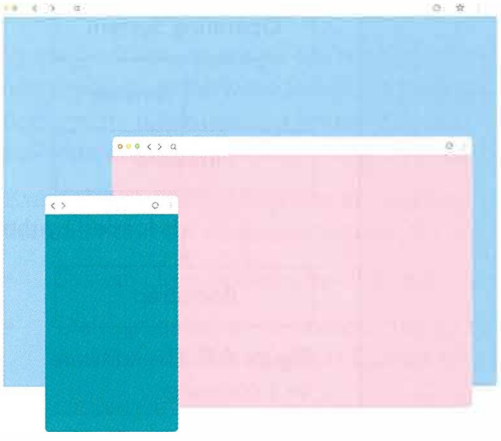


**Figure 4.6:** A Graphical User Interface

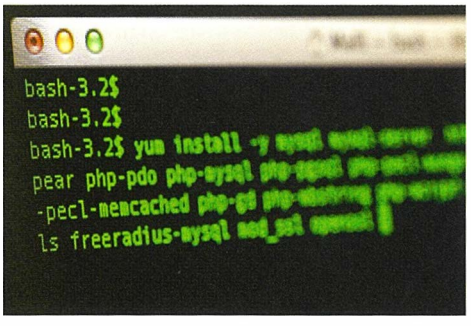



**Figure 4.7:** An image of a command prompt

**KEY WORDS**

**interface:** the method by which a user communicates with a computer.

**Graphical User Interface (GUI):** a type of operating system that includes windows, icons, menus and pointers.

**command line interface:** a type of operating system where the user types commands.

**natural language interface:** a type of operating system that allows the user to type or speak commands.

**COMPUTER SCIENCE IN CONTEXT**

Have you every spoken a command to a computer? For example, Amazon's Alexa, Microsoft's Cortana or Apple's Siri? These are examples of natural language interfaces. You do not have to speak specific commands, the programs analyse the words you have said and decide on a response. There is more about how this is done in Chapter 6.

# Managing files

A user will use application software to create files. The OS allows the creation of directories that the files can be stored within. It also allows the user to move files, copy files, delete files, name and rename files, plus many other actions such as sorting by date.

# Managing peripherals and drivers

A computer cannot be used without input and output devices. These are called **peripherals**. Anyone can build a peripheral, and it will be programmed by the person

**KEY WORD**

**peripheral:** a hardware device, used to input, store or output data from a computer, that is not directly part of the computer itself.

who made it – not the developer of the OS. This means that it may not be compatible (for example, it may use a different interpretation of binary). A **driver** is software that translates data from the computer to the peripheral, and vice-versa. The OS allows the installation of these drivers and the sending of data to and from it.

KEY WORD

**driver:** a program that controls a device, for example, a printer or a keyboard.

## Managing memory

A computer uses memory to store data. This can be using secondary or primary memory.

The OS manages the movement of data to and from hardware, primarily RAM. It also checks the processes being carried out and makes sure that they have enough memory allocated to perform their function. The OS will also make sure that two processes do not try and access the same memory location.

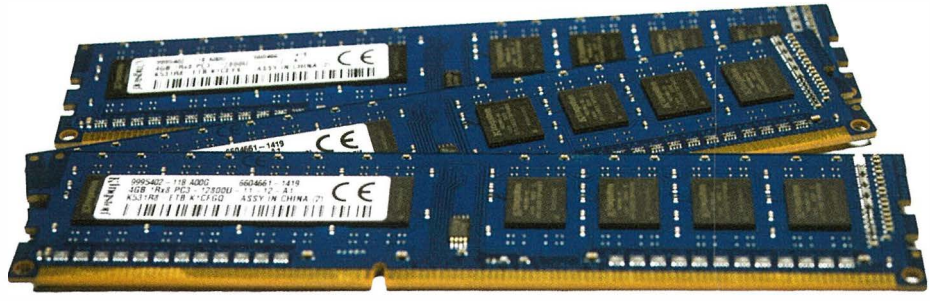


**Figure 4.8:** RAM memory modules

## Managing multitasking

A single processor can only ever execute one instruction at a time. The processor does this so fast that it appears to be doing several tasks at once. For example, you might be listening to music from a computer, typing a document into a word processor, and the computer is outputting what you are doing. The OS allows this to happen by deciding which processes should be executed next and how long they can spend being processed before switching to another process. The OS does this using **interrupts**.

KEY WORD

**interrupt:** a signal sent to the processor to tell it that its attention is required.

## Managing interrupts

An interrupt is a signal sent to the processor to tell it that something needs its attention. The OS decides which interrupt to run and then performs the actions required. There is more about interrupts in Section 4.3.

## Providing a platform for running applications

Application software needs to run on a computer. The software could be written by anyone using any language. The OS allows the application software to run on the computer by fetching instructions from it and executing them.

## Managing user accounts

Most computers need a user to set up an account. This can include preferences, as well as a username and password (text and/or biometric). The OS allows a user to set-up an account, keeps the data separate for multiple accounts, and restricts the access by using the password.

# 4.3 Interrupts

An interrupt is a signal to the processor to tell it that something needs its attention. An interrupt can be software or hardware based. Table 4.1 has several examples of each.

| Software | Hardware |
|---|---|
| Division by 0. | Data input, e.g. key pressed on a keyboard, mouse click. |
| Two processes attempting to access the same memory location. | Error from hardware, e.g. printer out of paper. |
| Program request for input. | Hardware failure. |
| Output required. | Hard drive signal that it has read data. |
| Data required from memory. | New hardware device connected. |

**Table 4.1:** Interrupt examples

An interrupt has a priority level. A high-priority interrupt needs the attention quickly, for example, hardware failure. A low-priority interrupt needs the processor but it's not urgent, for example, data input. This is managed by the **interrupt handler (IH)**.

When an interrupt is generated it is put in a queue according to its priority. The sequence of activities is:

- When the processor finishes its current fetch-decode-execute cycle (or before the starting the next FDE cycle), it checks the interrupt queue.
- It checks whether there is an interrupt with a higher priority than the current task it is processing.
- If it is, it:
  - Stores the current process and fetches the interrupt.
  - Checks the source of the interrupt.
  - Calls the relevant **interrupt service routine (ISR)**. This is a sequence of instructions that handle the interrupt.
  - When finished the stored process is returned to memory or another higher-priority interrupt is fetched.
- If it isn't, it runs another FDE cycle.

**KEY WORDS**

**interrupt handler (IH):** a program that organises interrupts into an order based upon priorities.

**interrupt service routine (ISR):** a program that retrieves an interrupt and performs the required action(s).

## Questions

1 Give **two** examples of application software.
2 State the purpose of system software.
3 Describe the relationship between application software and the bootstrap.
4 Identify **three** tasks the file management of an Operating System can perform.
5 Give **two** examples of software interrupts, and **two** examples of hardware interrupts.
6 Describe the process that happens when an interrupt is sent to the processor.

# 4.4 Types of programming language

When you are writing computer programs you will make use of specific software that allows you to write, edit, test and run the programs you write.

Programming languages can be categorised as **high-level** and **low-level**.

## High-level

A high-level language uses human-language style words. These could be English words that we recognise and use day-to-day. For example, `if`, `while`, `output`, `print`, `input`; as people we understand these, but they are also high-level language instructions. You will be learning to use a high-level language, for example, Python, Java and VB.NET.

Here are some examples of high-level language statements:

**Python:**

```
print("This is a high-level language statement")
```

**VB.NET:**

```
Dim x as integer
x = Console.readline()
```

**Java:**

```
num1 = 10;
num2 = 5;
num3 = num1 + num2;
```

A program that is written in a high-level language is considered **portable**; you can write a program on one computer, and run it on a different computer.

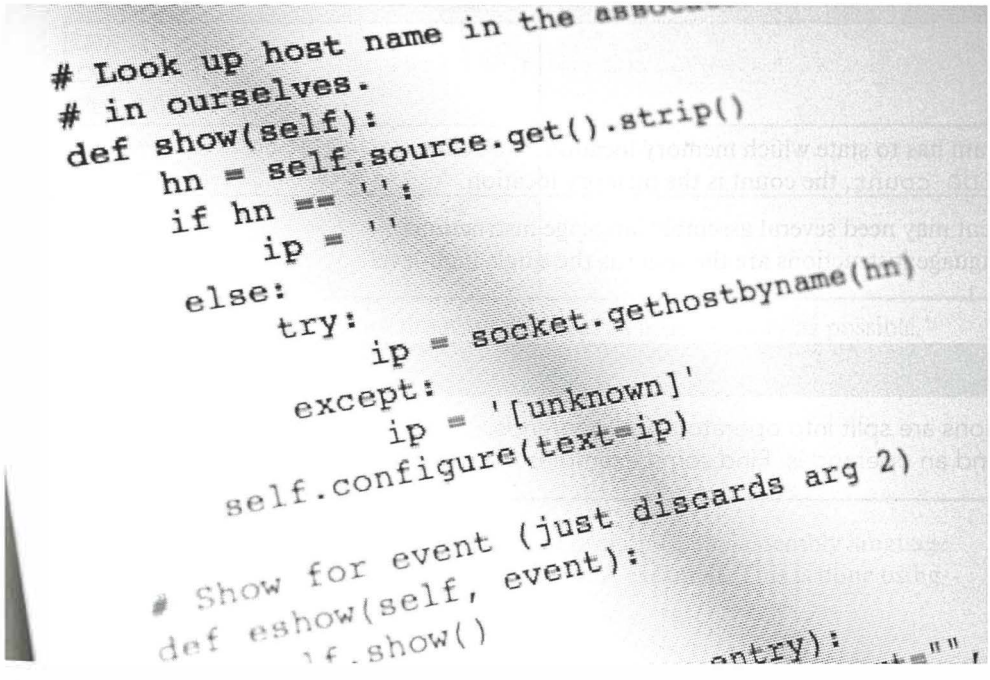



Figure 4.9: Code written in a high-level language

KEY WORDS

**high-level language:** a type of programming language that uses English-like commands.

**low-level language:** a language closer to the machine's language, this could be assembly language or binary code.

**portable:** a program that can be run on different types and manufacturers of computers.

## Low-level

There are two main types of low-level language, **machine code** and **assembly language**.

A computer executes machine code, this is in binary (1s and 0s). If you write a program in a high-level language it needs to be converted into machine code before it can executed. Different computers may have different machine code (for example, depending on the manufacturer). This means that machine code is **non portable**; you can write a program in machine code and it may not run on a different computer.

An example of machine code is: `01100110`

Assembly language is an in-between stage. It uses **mnemonics** to represent code, for example, to store data in a memory location it may use the mneumonic `STO`. This is a mid-way stage. Each high-level language statement is converted into assembly language instructions, which are then converted into machine code.



**Figure 4.10:** Machine code

Examples of assembly language are:

```
LDD count
ADD 1
STO count
```

In assembly language, the program has to state which memory locations are being used. In the previous example, `LDD count`, the count is the memory location.

One high-level language statement may need several assembly language instructions. The previous three assembly language instructions are the same as the single high-level language `count = count + 1`.

### ACTIVITY 4.2

Assembly language instructions are split into operators and operands. Find out what an operator and an operand is. Find some examples of each.

### KEY WORDS

**machine code:** binary code, an example of a low-level language.

**assembly language:** code written in mnemonics that allows direct manipulation of the hardware. It must be converted into binary code to run.

**non portable:** a program that cannot be run on different types and manufacturers of computers.

**mnemonic:** instruction code used in assembly language.

## Choosing a language

High- and low-level languages have their own benefits and drawbacks. Some of these are shown in Table 4.2.

| High-level language | Low-level language |
|---|---|
| Easier for users (humans) to understand, read, write, amend. | More difficult for users to understand, read, write and amend. |
| Easier for users (humans) to debug code. | Harder for users to debug code. |
| Portable, machine independent. | Not portable, machine dependent. |
| It must be converted into a low-level language before it can be run. | Machine code does not need converting.<br>Assembly language must be assembled before execution, but this is faster than from a high-level language. |
| One statement can represent many low-level instructions. | Several instructions are needed for each high-level language statement. |
| Cannot directly manipulate the hardware. | Can directly manipulate the hardware, e.g. writing to specific memory locations. This can make the program more efficient in terms of speed and memory usage. |

**Table 4.2:** Benefits and drawbacks of high- and low-level languages

## Questions

7 Describe what is meant by a high-level language.

8 Three examples of program code are given. Identify whether each example is a high-level language, assembly language, or machine code.

| Program code | High-level, assembly or machine code |
|---|---|
| `LDX 10`<br>`INC`<br>`STO 10` | |
| `value1 = input("Enter value")`<br>`print(value1 + 1)` | |
| `01100011`<br>`11000011`<br>`10101010` | |

9 Harry needs to write a utility program that uses as little memory as possible. Give the most appropriate type of programming language Harry should use and justify your choice.

# 4.5 Translators

In Section 4.4 you learnt that high-level language programs, and assembly language programs, need to be converted to machine code to be executed. This is done using a **translator**.

KEY WORD

**translator:** a type of software that converts code written in one programming language into another, usually a high-level language into a low-level language.

## Assembler

If you have a program written in assembly language, then an **assembler** is used to convert the instructions into machine code for the processor the code is being run on.

## Interpreters and compilers

If you have written a program in a high-level language then you can use an **interpreter** or a **compiler** to translate the instructions.

An interpreter:

- reads one line of the high-level language code
- checks that it is syntactically correct:
  - if it is not, it stops and reports an error to the user
  - If it is, it executes the statement and moves to the next.

Interpreters are most useful when writing the program, because it will stop at an error, allow you to correct it, and then continue executing from the same position.

Interpreters are not as useful for whole-system testing, or when the program is complete. This is because all of the code needs to be interpreted every time the program is run, even if there are no errors.

A compiler checks all the code, by going through one-line after the other. If there are any syntax errors, they are all reported to the user and the program is not executed. If there are no errors, then an **executable file** is created. An executable file uses intermediate or object code derived from the high-level language code. The actual program code is not included so if someone has the executable file, they cannot view or amend the original code. The executable file can be run without having to re-translate the code.

Compilers are useful when you have finished the program or need to test a working element repeatedly. This is because the code does not need to be translated each time, the executable file is run repeatedly.

Table 4.3 shows the comparisons of interpreters and compilers.

**KEY WORDS**

**assembler:** converts assembly language into machine code.

**interpreter:** a translator that converts a high-level language into a low-level language. It checks one line of code and then executes it before checking the next.

**compiler:** a translator that converts a high-level language into a low-level language. It checks all code before running the program.

**executable file:** a file produced by a compiler that can be run independent of translator software and does not include the source code.

| Interpreter | Compiler |
|---|---|
| Translates one line of code into machine code and then executes (runs) it. | Translates all lines of code into machine code, before executing the program. |
| Reports a syntax error as soon as it is picked up, and stops the program running until corrected. | Reports all syntax errors at the same time, the program is not run until all errors are corrected. |
| Useful when writing a program. | Useful when a program has been finished and is ready for testing or distribution. |
| Code needs to be re-translated each time the program is run. | Code does not need re-translating each time the program is run. |
| Does not produce an executable file. | Produces an executable file. |
| Source code is required to run the program. | Source code is not required to run the program, only the executable file. |

(continued)

| Interpreter | Compiler |
|---|---|
| Interpreter software is required to run the program. | No other software is required to run the executable file. |
| You can test part of a program without completing it all. | You need to have finished a section of code before testing it. |

**Table 4.3:** Comparison of interpreters and compilers

## Examples:

Saria is writing a computer game program that allows a character to move through different worlds collecting coins.

Which translator should Saria use when writing the code?

- It is more appropriate to use an interpreter. This will allow her to see when an error is identified, correct it and then continue running the program from the same position.
- A compiler can be used, but then all the errors would appear at the start and she would need to correct all of them before seeing how any of her program runs.
- An interpreter is considered more appropriate, but you could justify the use of a compiler in this situation.

Which translator should Saria use once she has finished programming the computer game and wants to sell it to users?

- A compiler should be used, because she can produce an executable file. There should be no syntax errors at this point, so correcting the code line-by-line is not required. The executable file can be distributed to people and they can then run the program without need to retranslate the instructions every time. The executable file also means that no-one will be able to see her code, so it is protected.
- An interpreter would not be an appropriate choice here, because the end user would need the interpreter software and the source code, and will need to wait for it to be interpreted each time. It also means the users can access the source code, and then copy it or change it.

### ACTIVITY 4.3

Write a computer program in a language that uses an interpreter, and one that uses a compiler. You might need to use a different programming language for each one. For example, Python usually has interpreters, whilst VB.NET and Java usually have compilers. Which translator did you prefer to use?

**Peer Assessment**

Discuss the reasons why you preferred the translator you chose. Did anyone give you some reasons that you hadn't thought of? Can you expand your own answer any further?

SKILLS FOCUS 4.1

## JUSTIFYING CHOICE

In an exam you may need to make a decision and then justify your choice. To answer this type question you will need to consider the context of the question and how each of the possible choices will impact the context. When you are justifying your decision you need to relate every reason back to the scenario.

Consider the choice of translator.

- An interpreter is most useful during the development of a program because:
  - you can fix errors in real time
  - the program will stop at each error to allow you to fix it
  - you can run small sections of the program when incomplete.
- A compiler is most useful when a program is complete because:
  - it produces a .exe file
  - the program does not need to be interpreted each time it is run
  - the program can be distributed without the source code.

**Questions**

Saria is writing a computer program.

1 Why is it more appropriate for Saria to use an interpreter while writing a program?

2 Why is it more appropriate for Saria to use a compiler when the program is complete?

# 4.6 Integrated Development Environment (IDE)

When you write a computer program you will most likely use an **Integrated Development Environment (IDE)**. This is a piece of software that includes an **editor** for you to enter and amend your code. This editor may include other features such as:

- **Auto-completion**; you start typing a command and it suggests options for you to select the correct one.
- **Auto-correction**; if you spell a command incorrect it changes it to what you meant to type.
- **Prettyprint**; this changes the colour of words, e.g. commands and identifiers. This helps you spot key terms and whether you have entered them correctly or not.
- Block minimising; you can make sections of code, e.g. procedures, minimised so there is less code to read through.

The IDE will also include the relevant translator, that you can select to translate the code. It will also provide a **run-time environment**; this is where the outputs from the executed code is displayed, it can be a command line or GUI depending on your language and the code entered.

KEY WORDS

**Integrated Development Environment (IDE):** a piece of software that allows a user to write, test and run program code.

**editor:** a feature of an IDE that allows the user to enter and amend program code.

**auto-completion:** a feature of an editor that gives the user options when they start typing a command.

**auto-correction:** a feature of an editor that identifies spelling errors and changes them.

**prettyprint:** a feature of an editor that changes the colour of text, for example, to highlight key words.

**run-time environment:** a feature of an IDE that allows a program to be run and lets the user interact with the program.

An IDE can also provide debugging features, or error diagnostics, these help you find the problems in your code. Some examples are:

- Break points; the user sets these points and the code stops running on that line. This lets you check the values and output from the code to that point.
- Variable watch window; the values of the variables are displayed while the code is running, so they can be checked whether they are correct.
- Stepping; the program will run one line at a time, and the user has to tell the program to move on to the next line, so it can be checked line by line.

You can write and run programs without an IDE. You can write your code in a basic text file without any formatting, and then run it (usually) through a command line interface by entering the correct instructions. An IDE is much more user friendly, with lots of tools to help you write, amend and correct your code.

### COMPUTER SCIENCE IN CONTEXT

You may not always get a choice of whether to use an interpreter or a compiler. If you use an IDE (Integrated Development Environment) it will probably come with one, or the other. If your IDE has an interpreter and you want to make an executable then you might need to download another IDE to compile it.

### ACTIVITY 4.4

Open up the IDE that you use to write programs. Identify the features that are mentioned in this chapter. Are there any different ones? If so, find out what these do.

## Questions

10 Name **three** types of translator.

11 Nasrat is writing a program in a high-level language that she wants to sell to make money. Identify when Nasrat could use an interpreter, and when she could use a compiler. Justify your decisions.

12 Describe **two** features of an IDE that help the user to test the program they have written.

### SELF ASSESSMENT

Question 11 required you to justify your choice. Did you give generic points, or did you relate each point you made to the scenario in the question? When a context is given, always try and go back to the question each time.

### REFLECTION

After completing Activity 4.4, it can be helpful to think about how you approached it and ways you could improve your research skills for future activities.

1 Discuss how you approached the investigation of the features. Did you test out the features, or search the internet to find out what they did? Do you think your approach was the most appropriate? Would you try a different way if the activity was repeated?

2 Discuss how this activity has helped you to identify the features. Does practically using them help you to remember them?

## SUMMARY

| |
|---|
| Software is created using programming languages. |
| Two types of software are application and system software. |
| The Operating System performs many functions including memory management, handling interrupts and providing an interface. |
| An interrupt is a signal sent to the processor to get its attention. |
| A high-level language uses English-like terms that need translating before it can run and a low-level language is assembly or machine code. |
| Assembly language is turned into machine code using an assembler.<br>High-level languages are turned into a low-level language using a compiler or interpreter. |
| An IDE helps the user write, test and run program code. |

## EXAM-STYLE QUESTIONS

**1** Tick *one* box in each row to identify if the software is an example of system software or application software.

| | System software | Application software |
|---|---|---|
| Word processor | | |
| Defragmentation | | |
| Operating System | | |
| Image manipulation | | |
| System clean-up | | |
| Database | | |
| Web browser | | |
| Spreadsheet | | |

**[5]**

**2** **State** what is meant by an interrupt. **[2]**

**3** **Describe** the difference between system software and application software using an example of each. **[3]**

**4** An operating system performs memory management.

**a** Describe memory management. **[3]**

**b** **Give** *two* other functions of an operating system. **[2]**

**[Total: 5]**

**5** A printer sends an interrupt to the computer to say that it is out of paper. Describe the stages involved in the processor handling the interrupt. **[5]**

**6** The following sentence describes the requirements for application software. Fill in the missing gaps.

The application software runs on the ..... ..... , this runs on the firmware, and it is loaded by the ..... **[2]**

### COMMAND WORDS

**state:** express in clear terms.

**describe:** state the points of a topic / give characteristics and main features.

**give:** produce an answer from a given source or recall / memory.

## CONTINUED

**7** Qui is writing a program using a high-level language.

**a** Describe what is meant by low-level language, and high-level language. **[4]**

**b** **Explain** why a person will usually write a program using a high-level language and not a low-level language. **[3]**

**c** Give *one* reason why a person might prefer to write a program using a low-level language. **[1]**

**d** Qui needs to use a translator while writing the program.

State what type of translator Qui should use, justify your choice. **[3]**

**e** Qui uses the translator in his Integrated Development Environment (IDE).

Describe the other features of an IDE that Qui can use to write and test his program. **[6]**

**[Total: 17]**

**8** Explain the role of the operating system in a computer. **[8]**

### COMMAND WORD

**explain:** set out purposes or reasons / make the relationships between things evident / provide why and/or how and support with relevant evidence.

## SELF-EVALUATION CHECKLIST

After studying this chapter, think about how confident you are with the different topics.
This will help you to see any gaps in your knowledge and help you to learn more effectively.

You might find it helpful to rate how confident you are for each of these statements when you are revising.
You should revisit any topics that you rated 'Needs more work' or 'Getting there'.

| I can... | See topic | Needs more work | Getting there | Confident to move on |
|---|---|---|---|---|
| describe the difference between application and system software. | 4.1 | | | |
| describe the role and basic functions of an operating system. | 4.2 | | | |
| describe how hardware, firmware and an operating system are required to run application software. | 4.2 | | | |
| describe the role and operation of interrupts. | 4.3 | | | |
| describe the characteristics, advantages and disadvantages, of high-level and low-level languages. | 4.4 | | | |
| describe the use of an assembler, compiler and interpreter. | 4.5 | | | |
| describe the advantages and disadvantages of a compiler and an interpreter. | 4.5 | | | |
| describe the use of, and features of, an IDE. | 4.6 | | | |